

Pandas

July 9, 2026

PAO25_25_07_Python_Pandas

July 08, 2026

PAO25-25 - Pandas

Kevin Xavier Suasnavas Villarreal

[Link a mi GitHub](#)

1 1 Pandas

- **Pandas** es la librería estándar en Python para trabajar con datos tabulares de forma eficiente.
 - Permite almacenar diferentes tipos de datos en una misma estructura (texto, enteros, decimales, valores booleanos, entre otros).
 - Sus dos estructuras principales son:
 - **Series:** estructura unidimensional.
 - **DataFrame:** estructura bidimensional o multidimensional, similar a una hoja de cálculo.
-

1.1 Importación de la librería Pandas

```
[2]: # =====  
# Importación de la librería Pandas  
# =====  
  
# Se importa la librería principal de Pandas.  
import pandas as pd  
  
# Se importan directamente las clases Series y DataFrame,  
# que son las estructuras de datos más utilizadas.  
from pandas import Series, DataFrame
```

1.2 Versión instalada de Pandas

Es recomendable verificar la versión instalada de la librería para asegurar la compatibilidad con los ejemplos que se desarrollarán durante la práctica.

```
[3]: # =====  
# Mostrar la versión instalada de Pandas  
# =====  
  
# Muestra la versión de Pandas instalada en el entorno.  
pd.__version__
```

```
[3]: '3.0.3'
```

2 2 Series

Las **Series** son estructuras de datos unidimensionales proporcionadas por Pandas.

Sus principales características son:

- Almacenan una única columna de datos.
- Cada elemento posee un índice asociado.
- Tanto los datos como los índices pueden modificarse.
- Son similares a un arreglo de NumPy, pero ofrecen un manejo más flexible de los índices.

2.1 2.1 Creación de Series

Una **Serie (Series)** es una estructura unidimensional de Pandas que almacena una colección de datos junto con un índice.

Por defecto, Pandas asigna índices numéricos consecutivos comenzando desde **0**, aunque es posible personalizar dichos índices para facilitar la identificación de los datos.

```
[4]: # =====  
# Crear una Serie utilizando los índices predeterminados  
# =====  
  
# Se crea una Serie con varios países.  
countries = pd.Series([  
    'Spain',  
    'Andorra',  
    'Gibraltar',  
    'Portugal',  
    'France'  
)  
  
# Mostrar la Serie completa.  
print(countries)
```

```
0      Spain
1      Andorra
2      Gibraltar
3      Portugal
4      France
dtype: str
```

2.2 Series con índices personalizados

Además de utilizar índices automáticos, Pandas permite definir índices personalizados, lo que facilita localizar la información utilizando identificadores específicos.

```
[5]: # =====
# Crear una Serie con índices personalizados
# =====

# Se crea una Serie indicando manualmente los índices.
countries = pd.Series(
    ['Spain', 'Andorra', 'Gibraltar', 'Portugal', 'France'],
    index=range(10, 60, 10)
)

# Mostrar la Serie.
print(countries)
```

```
10      Spain
20      Andorra
30      Gibraltar
40      Portugal
50      France
dtype: str
```

2.3 Series con índices de tipo texto

Los índices no necesariamente deben ser números. También pueden ser cadenas de texto, caracteres u otros tipos de datos.

```
[6]: # =====
# Crear una Serie utilizando índices de texto
# =====

# Se crea una Serie donde los índices son letras.
football_cities = pd.Series(
    ['Barcelona', 'Madrid', 'Valencia', 'Sevilla'],
    index=['a', 'b', 'c', 'd']
)

# Mostrar la Serie.
print(football_cities)
```

```
a    Barcelona
b      Madrid
c    Valencia
d      Sevilla
dtype: str
```

2.4 Asignación de atributos

Las Series pueden tener un nombre descriptivo y también es posible asignar un nombre a su índice para facilitar la interpretación de la información.

```
[7]: # =====
# Asignar nombres a la Serie y a sus índices
# =====

# Asignar un nombre descriptivo a la Serie.
football_cities.name = "Ciudades con dos equipos en primera"

# Asignar un nombre al índice.
football_cities.index.name = "Id"

# Mostrar la Serie con sus nuevos atributos.
print(football_cities)
```

```
Id
a    Barcelona
b      Madrid
c    Valencia
d      Sevilla
Name: Ciudades con dos equipos en primera, dtype: str
```

2.5 Acceso a los elementos de una Serie

Los elementos de una Serie pueden recuperarse utilizando:

- La posición del elemento.
- El valor del índice asignado.

Ambos métodos permiten acceder al mismo dato cuando corresponden al mismo elemento.

```
[9]: # =====
# Acceder a los elementos de una Serie
# =====

# Acceso utilizando la posición (tercer elemento).
print(football_cities.iloc[2])

# Acceso utilizando el índice semántico.
print(football_cities.loc['c'])
```

```
# Comprobar que ambos accesos devuelven el mismo valor.
print(football_cities.iloc[2] == football_cities.loc['c'])
```

```
Valencia
Valencia
True
```

3 3 Tratamiento similar a ndarray

Las **Series** de Pandas comparten muchas características con los arreglos de **NumPy**. Es posible seleccionar varios elementos, realizar operaciones de slicing, convertir una Serie a otros tipos de datos y aplicar funciones de NumPy sobre ella.

3.1 Selección de múltiples elementos

Una Serie permite recuperar varios elementos simultáneamente indicando una lista con los índices o posiciones que se desean obtener.

```
[10]: # =====
# Seleccionar múltiples elementos de una Serie
# =====

# Selección utilizando los índices de la Serie.
print(football_cities[['a', 'c']])

# Selección utilizando posiciones mediante iloc.
print(football_cities.iloc[[0, 3]])
```

```
Id
a    Barcelona
c    Valencia
Name: Ciudades con dos equipos en primera, dtype: str
Id
a    Barcelona
d    Sevilla
Name: Ciudades con dos equipos en primera, dtype: str
```

3.2 Slicing

Al igual que las listas de Python, las Series permiten obtener subconjuntos mediante **slicing**.

Cuando se utilizan índices semánticos con `.loc`, el extremo final también se incluye.

```
[11]: # =====
# Ejemplos de slicing en Series
# =====

# Slicing utilizando índices semánticos.
print(football_cities.loc[:'c'])
```

```
# Slicing utilizando posiciones.
print(football_cities.iloc[:2])
```

```
Id
a    Barcelona
b      Madrid
c    Valencia
Name: Ciudades con dos equipos en primera, dtype: str
Id
a    Barcelona
b      Madrid
Name: Ciudades con dos equipos en primera, dtype: str
```

3.3 Conversión de una Serie a lista

Una Serie puede convertirse fácilmente en una lista de Python utilizando la función `list()`.

```
[12]: # =====
# Convertir una Serie a lista
# =====

lista = list(football_cities.loc[:'c'])

print(lista)
print(type(lista))

['Barcelona', 'Madrid', 'Valencia']
<class 'list'>
```

3.4 Tipo del objeto resultante

Aunque se realice un slicing, el resultado continúa siendo una **Serie** de Pandas.

```
[13]: # =====
# Comprobar el tipo de dato
# =====

print(type(football_cities.loc[:'c']))

<class 'pandas.Series'>
```

3.5 Conversión a un arreglo de NumPy

Una Serie puede transformarse en un arreglo (`ndarray`) para aprovechar todas las funciones de NumPy.

```
[14]: # =====
# Convertir una Serie a ndarray
# =====
```

```
import numpy as np

cities = np.array(football_cities.loc[:'c'])

print(cities)
print(type(cities))
```

```
['Barcelona' 'Madrid' 'Valencia']
<class 'numpy.ndarray'>
```

3.6 Conversión a diccionario

También es posible convertir una Serie en un diccionario donde los índices se convierten en las claves y los valores permanecen asociados.

```
[15]: # =====
# Convertir una Serie a diccionario
# =====

diccionario = dict(football_cities.loc[:'c'])

print(diccionario)
```

```
{'a': 'Barcelona', 'b': 'Madrid', 'c': 'Valencia'}
```

3.7 Selección mediante máscaras

Las máscaras permiten filtrar únicamente aquellos elementos que cumplen una determinada condición lógica.

```
[16]: # =====
# Filtrado utilizando máscaras booleanas
# =====

# Crear una Serie con la sucesión de Fibonacci.
fibonacci = pd.Series([0, 1, 1, 2, 3, 5, 8, 13, 21])

print(fibonacci)

# Crear una máscara con los valores mayores que 10.
mask = fibonacci > 10

print(mask)

# Mostrar únicamente los elementos que cumplen la condición.
print(fibonacci[mask])

# Serie esperada para realizar la comparación.
```

```

dst = pd.Series([13, 21])

print(dst)

# Comparación inicial (devuelve False porque los índices son diferentes).
print(dst.equals(fibonacci))

# Reiniciar los índices de la Serie filtrada.
fb = fibonacci[mask]
fb.reset_index(drop=True, inplace=True)

print(fb)

# Ahora ambas Series son iguales.
print(dst.equals(fb))

```

```

0      0
1      1
2      1
3      2
4      3
5      5
6      8
7     13
8     21
dtype: int64
0      False
1      False
2      False
3      False
4      False
5      False
6      False
7       True
8       True
dtype: bool
7     13
8     21
dtype: int64
0     13
1     21
dtype: int64
False
0     13
1     21
dtype: int64
True

```

3.8 Aplicación de funciones de NumPy

Las funciones de NumPy pueden utilizarse directamente sobre una Serie de Pandas.

```
[17]: # =====  
# Aplicar funciones de NumPy sobre una Serie  
# =====  
  
import numpy as np  
  
print(np.sum(fibonacci))
```

54

3.9 Filtrado utilizando np.where

La función `np.where()` permite reemplazar valores de una Serie dependiendo de si cumplen o no una determinada condición.

```
[18]: # =====  
# Reemplazar valores nulos utilizando np.where  
# =====  
  
distances = pd.Series([12.1, np.nan, 12.8, 76.9, 6.1, 7.2])  
  
# Sustituir los valores NaN por el número 2.  
valid_distances = np.where(pd.notnull(distances), distances, 2)  
  
print(valid_distances)  
print(type(valid_distances))
```

```
[12.1  2.  12.8 76.9  6.1  7.2]  
<class 'numpy.ndarray'>
```

4 3.0.1 Iteración

Las **Series** de Pandas permiten recorrer fácilmente tanto sus valores como sus índices.

Existen varias formas de iterar sobre una Serie:

- Recorrer únicamente los valores.
- Recorrer únicamente los índices.
- Recorrer índices y valores simultáneamente mediante `items()`.
- Utilizar `zip()` para combinar índices y valores durante la iteración.

Estas opciones son muy útiles cuando se necesita procesar o mostrar la información almacenada en una Serie.

```
[19]: # =====  
# Iterar sobre una Serie de Pandas  
# =====
```

```

import pandas as pd

# Crear una Serie con la sucesión de Fibonacci.
fibonacci = pd.Series([0, 1, 1, 2, 3, 5, 8, 13, 21])

# -----
# Recorrer únicamente los valores de la Serie.
# -----
print("Recorrido de valores:")

for value in fibonacci:
    print("Valor:", value)

print()

# -----
# Recorrer únicamente los índices de la Serie.
# -----
print("Recorrido de índices:")

for index in fibonacci.index:
    print("Índice:", index)

```

Recorrido de valores:

```

Valor: 0
Valor: 1
Valor: 1
Valor: 2
Valor: 3
Valor: 5
Valor: 8
Valor: 13
Valor: 21

```

Recorrido de índices:

```

Índice: 0
Índice: 1
Índice: 2
Índice: 3
Índice: 4
Índice: 5
Índice: 6
Índice: 7
Índice: 8

```

```
[20]: # =====
# Recorrer índices y valores al mismo tiempo utilizando items()
# =====

for index, value in fibonacci.items():
    print(f"Índice: {index} Valor: {value}")
```

```
Índice: 0 Valor: 0
Índice: 1 Valor: 1
Índice: 2 Valor: 1
Índice: 3 Valor: 2
Índice: 4 Valor: 3
Índice: 5 Valor: 5
Índice: 6 Valor: 8
Índice: 7 Valor: 13
Índice: 8 Valor: 21
```

```
[21]: # =====
# Otra forma de recorrer índices y valores utilizando zip()
# =====

for index, value in zip(fibonacci.index, fibonacci):
    print(f"Índice: {index} Valor: {value}")
```

```
Índice: 0 Valor: 0
Índice: 1 Valor: 1
Índice: 2 Valor: 1
Índice: 3 Valor: 2
Índice: 4 Valor: 3
Índice: 5 Valor: 5
Índice: 6 Valor: 8
Índice: 7 Valor: 13
Índice: 8 Valor: 21
```

5 3.1 Series como diccionarios

Una **Serie** de Pandas puede comportarse de manera muy similar a un diccionario de Python.

En este caso:

- Los índices actúan como las claves.
- Los valores almacenan la información asociada.
- Es posible agregar, modificar y eliminar elementos utilizando sus índices.
- También es posible consultar si una clave existe antes de acceder a ella para evitar errores.

```
[22]: # =====
# Crear una Serie a partir de un diccionario
# =====
```

```

import pandas as pd

# Cada clave del diccionario será un índice de la Serie.
serie = pd.Series({
    "Carlos": 100,
    "Marcos": 98
})

# Mostrar información de la Serie.
print("Índices:")
print(serie.index)

print("\nValores:")
print(serie.values)

print("\nSerie completa:")
print(serie)

print("\nTipo del objeto:")
print(type(serie))

```

Índices:
Index(['Carlos', 'Marcos'], dtype='str')

Valores:
[100 98]

Serie completa:
Carlos 100
Marcos 98
dtype: int64

Tipo del objeto:
<class 'pandas.Series'>

```

[23]: # =====
# Modificar una Serie como si fuera un diccionario
# =====

# Agregar un nuevo elemento.
serie["Pedro"] = 12

# Modificar el valor existente.
serie["Pedro"] = 15

# Eliminar un elemento mediante su índice.
del serie["Marcos"]

```

```
print("Serie modificada:")
print(serie)
```

```
Serie modificada:
Carlos    100
Pedro     15
dtype: int64
```

```
[24]: # =====
# Verificar si un índice existe antes de acceder a él
# =====

# Evita errores si el índice no existe.
if "Marcos" in serie:
    print(serie["Marcos"])
else:
    print("El índice 'Marcos' no existe.")

print("\nSerie actual:")
print(serie)
```

```
El índice 'Marcos' no existe.
```

```
Serie actual:
Carlos    100
Pedro     15
dtype: int64
```

6 3.2 Operaciones entre Series

Las **Series** de Pandas permiten realizar operaciones matemáticas de forma automática entre sus elementos.

Cuando dos Series poseen los mismos índices, las operaciones se realizan elemento por elemento.

Si alguno de los índices no existe en ambas Series, el resultado para ese índice será **NaN** (*Not a Number*), indicando que no fue posible realizar la operación.

Las operaciones más comunes son la suma, resta, multiplicación y división entre Series.

```
[25]: # =====
# Suma de dos Series
# =====

import pandas as pd

# Crear la primera Serie.
serie1 = pd.Series([10, 20, 30, 40], index=range(4))
```

```

# Crear la segunda Serie.
serie2 = pd.Series([1, 2, 3], index=range(3))

# La suma se realiza utilizando los índices.
# Si un índice no existe en ambas Series, el resultado será NaN.
suma = serie1 + serie2

print("Serie 1:")
print(serie1)

print("\nSerie 2:")
print(serie2)

print("\nResultado de la suma:")
print(suma)

```

```

Serie 1:
0    10
1    20
2    30
3    40
dtype: int64

```

```

Serie 2:
0     1
1     2
2     3
dtype: int64

```

```

Resultado de la suma:
0    11.0
1    22.0
2    33.0
3     NaN
dtype: float64

```

```

[26]: # =====
# Resta entre dos Series
# =====

# La resta funciona igual que la suma:
# Pandas alinea automáticamente los elementos por su índice.
resta = serie1 - serie2

print("Resultado de la resta:")
print(resta)

```

Resultado de la resta:

```
0    9.0
1   18.0
2   27.0
3    NaN
dtype: float64
```

```
[27]: # =====
# Reemplazar valores NaN después de una operación
# =====

# Realizar nuevamente la suma.
resultado = serie1 + serie2

print("Resultado original:")
print(resultado)

# Identificar los elementos cuyo valor es NaN.
resultado[pd.isnull(resultado)] = 0

print("\nResultado después de reemplazar NaN por 0:")
print(resultado)
```

Resultado original:

```
0    11.0
1    22.0
2    33.0
3     NaN
dtype: float64
```

Resultado después de reemplazar NaN por 0:

```
0    11.0
1    22.0
2    33.0
3     0.0
dtype: float64
```

6.1 3.2.1 Diferencias entre Pandas Series y un diccionario

Aunque una **Serie** puede parecer similar a un diccionario de Python, existen diferencias importantes:

- Un **diccionario** relaciona claves y valores sin mantener una estructura posicional estricta.
- Una **Serie** organiza los datos mediante índices alineados con cada valor.
- Las Series permiten realizar operaciones matemáticas de forma automática entre elementos con el mismo índice.
- Los valores pueden almacenarse a partir de listas o arreglos de NumPy.
- Tanto los índices como los valores pueden ser numéricos o de texto.

- Las Series combinan características de las listas y los diccionarios, proporcionando una estructura eficiente para el análisis de datos en una dimensión.

7 4 DataFrame

Los **DataFrames** son la estructura de datos más utilizada en Pandas para trabajar con información tabular.

Un DataFrame está compuesto por filas y columnas, donde cada columna es una **Serie** que comparte el mismo índice con las demás.

Los DataFrames permiten almacenar diferentes tipos de datos (texto, enteros, decimales, booleanos, etc.) y facilitan su manipulación y análisis.

```
[29]: # =====
# Crear un DataFrame a partir de un diccionario
# =====

import pandas as pd

# Crear un diccionario donde cada clave representa una columna.
diccionario = {
    "Nombre": ["Marisa", "Laura", "Manuel"],
    "Edad": [34, 29, 12]
}

print("Diccionario original:")
print(diccionario)

# Crear el DataFrame.
frame = pd.DataFrame(diccionario)

print("\nDataFrame generado:")
display(frame)
```

Diccionario original:

```
{'Nombre': ['Marisa', 'Laura', 'Manuel'], 'Edad': [34, 29, 12]}
```

DataFrame generado:

	Nombre	Edad
0	Marisa	34
1	Laura	29
2	Manuel	12

```
[30]: # =====
# Crear un DataFrame indicando índices personalizados
# =====
```

```
# Los índices identifican cada fila.
frame = pd.DataFrame(
    diccionario,
    index=["a", "b", "c"]
)

display(frame)
```

	Nombre	Edad
a	Marisa	34
b	Laura	29
c	Manuel	12

```
[31]: # =====
# Especificar el orden de las columnas
# =====

# También es posible indicar columnas que aún no existen.
# En ese caso Pandas rellenará esos valores con NaN.
frame = pd.DataFrame(
    diccionario,
    columns=[
        "Nacionalidad",
        "Nombre",
        "Edad",
        "Profesion",
        "Genero"
    ]
)

display(frame)
```

	Nacionalidad	Nombre	Edad	Profesion	Genero
0	NaN	Marisa	34	NaN	NaN
1	NaN	Laura	29	NaN	NaN
2	NaN	Manuel	12	NaN	NaN

7.1 Acceso a columnas

Cada columna de un DataFrame puede recuperarse como una **Serie**.

Existen dos formas principales de acceder a una columna:

- Utilizando corchetes (`frame["Nombre"]`), que es la forma recomendada.
- Utilizando la notación por punto (`frame.Nombre`), siempre que el nombre de la columna sea un identificador válido.

```
[32]: # =====
# Acceder a columnas del DataFrame
```

```
# =====

# Obtener la columna "Nombre".
nombres = frame["Nombre"]

print("Columna Nombre:")
display(nombres)

print(type(nombres))

# Obtener la columna "Edad".
edades = frame["Edad"]

print("\nColumna Edad:")
display(edades)

print(type(edades))
```

Columna Nombre:

```
0    Marisa
1    Laura
2    Manuel
Name: Nombre, dtype: str
<class 'pandas.Series'>
```

Columna Edad:

```
0    34
1    29
2    12
Name: Edad, dtype: int64
<class 'pandas.Series'>
```

```
[33]: # =====
# Acceso utilizando la notación por punto
# =====

# Esta forma funciona únicamente cuando el nombre de la columna
# es un identificador válido.

profesiones = frame.Profesion

display(profesiones)

print(type(profesiones))
```

```
0    NaN
```

```
1    NaN
2    NaN
Name: Profesion, dtype: object

<class 'pandas.Series'>
```

```
[34]: # =====
# Acceder a un valor específico del DataFrame
# =====

# Obtener el primer nombre utilizando corchetes.
print(frame["Nombre"][0])

# Obtener el mismo valor utilizando la notación por punto.
print(frame.Nombre[0])

# Acceder al primer elemento de la Serie almacenada en la variable.
print(profesiones[0])
```

```
Marisa
Marisa
nan
```

7.2 4.0.1 Formas de crear un DataFrame

Pandas ofrece varias maneras de crear un **DataFrame**, dependiendo del origen de los datos.

Las formas más comunes son:

- A partir de una **Serie** de Pandas.
- A partir de una **lista de diccionarios**.
- A partir de un **diccionario de Series**.
- A partir de un **array bidimensional de NumPy**.
- A partir de un **array estructurado de NumPy**.

La elección depende de cómo estén organizados los datos antes de ser procesados.

8 4.1 Modificar DataFrames

Los DataFrames son estructuras dinámicas que permiten agregar, modificar o eliminar tanto filas como columnas.

Entre las operaciones más comunes se encuentran:

- Agregar nuevas columnas.
- Modificar el contenido de una columna existente.
- Insertar nuevas filas.
- Eliminar filas o columnas.
- Obtener la transpuesta del DataFrame.

```
[35]: # =====
# Agregar nuevas columnas a un DataFrame
# =====

import pandas as pd

# Crear el diccionario base.
diccionario = {
    "Nombre": ["Marisa", "Laura", "Manuel"],
    "Edad": [34, 29, 12]
}

# Crear el DataFrame indicando todas las columnas.
frame = pd.DataFrame(
    diccionario,
    columns=[
        "Nacionalidad",
        "Nombre",
        "Edad",
        "Profesion",
        "Direccion"
    ]
)

# Agregar una columna con el mismo valor para todas las filas.
frame["Direccion"] = "Desconocida"

display(frame)
```

	Nacionalidad	Nombre	Edad	Profesion	Direccion
0	NaN	Marisa	34	NaN	Desconocida
1	NaN	Laura	29	NaN	Desconocida
2	NaN	Manuel	12	NaN	Desconocida

```
[36]: # =====
# Asignar distintos valores a una columna
# =====

# Lista con una dirección para cada persona.
lista_direcciones = [
    "Rue 13 del Percebe, 13",
    "Evergreen Terrace, 3",
    "Av. de los Rombos, 12"
]

# Reemplazar la columna con los nuevos valores.
frame["Direccion"] = lista_direcciones
```

```
display(frame)
```

	Nacionalidad	Nombre	Edad	Profesion	Direccion
0	NaN	Marisa	34	NaN	Rue 13 del Percebe, 13
1	NaN	Laura	29	NaN	Evergreen Terrace, 3
2	NaN	Manuel	12	NaN	Av. de los Rombos, 12

```
[37]: # =====
# Agregar una nueva fila utilizando loc
# =====

# Cada elemento corresponde a una columna del DataFrame.
usuario_nuevo = [
    "Alemania",
    "Klaus",
    20,
    "None",
    "Desconocida"
]

# Insertar la nueva fila en la posición 3.
frame.loc[3] = usuario_nuevo

display(frame)
```

	Nacionalidad	Nombre	Edad	Profesion	Direccion
0	NaN	Marisa	34	NaN	Rue 13 del Percebe, 13
1	NaN	Laura	29	NaN	Evergreen Terrace, 3
2	NaN	Manuel	12	NaN	Av. de los Rombos, 12
3	Alemania	Klaus	20	None	Desconocida

```
[38]: # =====
# Eliminar filas y columnas
# =====

# Crear nuevamente el DataFrame.
frame = pd.DataFrame(
    diccionario,
    columns=[
        "Nacionalidad",
        "Nombre",
        "Edad",
        "Profesion"
    ]
)

# -----
# Eliminar una fila.
```

```

# drop() devuelve un nuevo DataFrame, por ello es necesario
# guardar nuevamente el resultado.
# -----
frame = frame.drop(2)

display(frame)

# -----
# Eliminar una columna.
# inplace=True modifica directamente el DataFrame.
# -----
frame.drop("Nombre", axis=1, inplace=True)

display(frame)

```

	Nacionalidad	Nombre	Edad	Profesion
0	NaN	Marisa	34	NaN
1	NaN	Laura	29	NaN

	Nacionalidad	Edad	Profesion
0	NaN	34	NaN
1	NaN	29	NaN

```

[39]: # =====
# Eliminar una columna utilizando del
# =====

# Eliminar completamente la columna "Profesion".
del frame["Profesion"]

display(frame)

```

	Nacionalidad	Edad
0	NaN	34
1	NaN	29

```

[40]: # =====
# Mostrar la transpuesta del DataFrame
# =====

# La propiedad T intercambia filas por columnas.
display(frame.T)

```

	0	1
Nacionalidad	NaN	NaN
Edad	34	29

8.0.1 Observaciones

- Para **agregar columnas**, basta con asignar un nuevo nombre de columna.

- Para **agregar filas**, se utiliza generalmente `loc`.
- El método **`drop()`** devuelve un nuevo DataFrame; por ello normalmente se reasigna el resultado o se utiliza `inplace=True`.
- La palabra clave **`del`** elimina definitivamente una columna.
- La propiedad **`T`** permite obtener la transpuesta del DataFrame, intercambiando filas por columnas.

9 4.2 Iteración

Los DataFrames permiten recorrer tanto sus columnas como sus filas.

Existen varias formas de iterar sobre un DataFrame:

- Recorrer los nombres de las columnas.
- Recorrer las filas mediante la propiedad `values`.
- Recorrer cada elemento individual utilizando bucles anidados.

La forma de iterar dependerá de la información que se necesite procesar.

```
[41]: # =====
# Iterar sobre las columnas de un DataFrame
# =====

import pandas as pd

# Crear un diccionario con los datos.
diccionario = {
    "Nombre": ["Marisa", "Laura", "Manuel"],
    "Edad": [34, 29, 12]
}

# Crear el DataFrame.
frame = pd.DataFrame(
    diccionario,
    columns=["Nacionalidad", "Nombre", "Edad", "Profesion"]
)

display(frame)

# -----
# Al recorrer directamente un DataFrame se obtienen los nombres
# de las columnas.
# -----
for columna in frame:
    print(columna)

print(type(columna))
```

```
Nacionalidad  Nombre  Edad  Profesion
```

0	NaN	Marisa	34	NaN
1	NaN	Laura	29	NaN
2	NaN	Manuel	12	NaN

Nacionalidad
Nombre
Edad
Profesion
<class 'str'>

```
[42]: # =====
# Iterar sobre las filas
# =====

# values devuelve un ndarray con todas las filas.
for fila in frame.values:
    print(fila)

print(type(frame.values[0]))
```

[nan 'Marisa' 34 nan]
[nan 'Laura' 29 nan]
[nan 'Manuel' 12 nan]
<class 'numpy.ndarray'>

```
[43]: # =====
# Recorrer cada elemento del DataFrame
# =====

# Primero se recorre cada fila y luego cada valor.
for fila in frame.values:
    for valor in fila:
        print(valor)
```

nan
Marisa
34
nan
nan
Laura
29
nan
nan
Manuel
12
nan

10 4.3 Indexación y slicing con DataFrames

Pandas ofrece dos formas principales para acceder a los datos de un DataFrame:

- `iloc[]`: acceso mediante posiciones numéricas.
- `loc[]`: acceso utilizando los nombres de filas y columnas.

Estas herramientas permiten seleccionar filas, columnas o regiones completas del DataFrame de forma sencilla y eficiente.

```
[45]: # =====  
# Crear un DataFrame para los ejemplos de indexación  
# =====  
  
d1 = {"ciudad": "Valencia", "temperatura": 10, "o2": 1}  
d2 = {"ciudad": "Barcelona", "temperatura": 8}  
d3 = {"ciudad": "Valencia", "temperatura": 9}  
d4 = {"ciudad": "Madrid", "temperatura": 10, "humedad": 80}  
d5 = {"ciudad": "Sevilla", "temperatura": 15, "humedad": 50, "co2": 6}  
d6 = {"ciudad": "Valencia", "temperatura": 10, "humedad": 90, "co2": 10}  
  
# Crear una lista de diccionarios.  
lista_datos = [d1, d2, d3, d4, d5, d6]  
  
# Crear el DataFrame indicando índices personalizados.  
df_data = pd.DataFrame(lista_datos, index=list("abcdef"))  
  
display(df_data)
```

	ciudad	temperatura	o2	humedad	co2
a	Valencia	10	1.0	NaN	NaN
b	Barcelona	8	NaN	NaN	NaN
c	Valencia	9	NaN	NaN	NaN
d	Madrid	10	NaN	80.0	NaN
e	Sevilla	15	NaN	50.0	6.0
f	Valencia	10	NaN	90.0	10.0

```
[46]: # =====  
# Acceder a datos utilizando iloc y loc  
# =====  
  
# Acceso por posición.  
print(df_data.iloc[1, 1])  
  
# Primeras filas y columnas utilizando posiciones.  
display(df_data.iloc[:3, :4])  
  
# Acceso por etiquetas.  
display(df_data.loc["a", "temperatura"])
```

```
display(df_data.loc[:,"c", :,"o2"])

display(df_data.loc[:,"c", "temperatura":,"o2"])

display(df_data.loc[:, ["ciudad", "o2"]])
```

8

```
      ciudad  temperatura  o2  humedad
a  Valencia          10  1.0      NaN
b  Barcelona          8  NaN      NaN
c  Valencia          9  NaN      NaN
```

np.int64(10)

```
      ciudad  temperatura  o2
a  Valencia          10  1.0
b  Barcelona          8  NaN
c  Valencia          9  NaN
```

```
      temperatura  o2
a          10  1.0
b           8  NaN
c           9  NaN
```

```
      ciudad  o2
a  Valencia  1.0
b  Barcelona NaN
c  Valencia NaN
d    Madrid NaN
e   Sevilla NaN
f  Valencia NaN
```

```
[47]: # =====
# Acceso a columnas
# =====

# Una columna devuelve una Serie.
print(df_data["ciudad"])

# Varias columnas devuelven un DataFrame.
display(df_data[["ciudad", "o2"]])
```

```
a  Valencia
b  Barcelona
c  Valencia
d    Madrid
e   Sevilla
f  Valencia
Name: ciudad, dtype: str
```

```

      ciudad  o2
a  Valencia  1.0
b  Barcelona NaN
c  Valencia  NaN
d   Madrid  NaN
e   Sevilla  NaN
f  Valencia  NaN

```

```

[49]: # =====
      # Intentar acceder utilizando un entero
      # =====

      # Esto produce un KeyError porque Pandas interpreta el 0
      # como el nombre de una columna y no como la primera fila.

df_data[0]

```

```

-----
KeyError                                Traceback (most recent call last)
File ~\anaconda3\envs\machine2026\Lib\site-packages\pandas\core\indexes\base.py
  ~3641, in Index.get_loc(self, key)
    3640 try:
-> 3641     return self._engine.get_loc(casted_key)
    3642 except KeyError as err:

File pandas\_libs\index.pyx:168, in pandas._libs.index.IndexEngine.get_loc()
--> 168 'Could not get source, probably due dynamically evaluated source code.'

File pandas\_libs\index.pyx:176, in pandas._libs.index.IndexEngine.get_loc()
--> 176 'Could not get source, probably due dynamically evaluated source code.'

File pandas\_libs\index.pyx:583, in pandas._libs.index.StringObjectEngine.
  ~_check_type()
--> 583 'Could not get source, probably due dynamically evaluated source code.'

KeyError: 0

```

The above exception was the direct cause of the following exception:

```

KeyError                                Traceback (most recent call last)
Cell In[49], line 8
      4
      5 # Esto produce un KeyError porque Pandas interpreta el 0
      6 # como el nombre de una columna y no como la primera fila.
      7
----> 8 df_data[0]

```

```

File ~\anaconda3\envs\machine2026\Lib\site-packages\pandas\core\frame.py:4378,
↳ in DataFrame.__getitem__(self, key)
    4374
    4375         if is_single_key:
    4376             if self.columns.nlevels > 1:
    4377                 return self._getitem_multilevel(key)
-> 4378             indexer = self.columns.get_loc(key)
    4379             if is_integer(indexer):
    4380                 indexer = [indexer]
    4381             else:

File ~\anaconda3\envs\machine2026\Lib\site-packages\pandas\core\indexes\base.py
↳ 3648, in Index.get_loc(self, key)
    3643         if isinstance(casted_key, slice) or (
    3644             isinstance(casted_key, abc.Iterable)
    3645             and any(isinstance(x, slice) for x in casted_key)
    3646         ):
    3647             raise InvalidIndexError(key) from err
-> 3648         raise KeyError(key) from err
    3649 except TypeError:
    3650     # If we have a listlike key, _check_indexing_error will raise
    3651     # InvalidIndexError. Otherwise we fall through and re-raise
    3652     # the TypeError.
    3653     self._check_indexing_error(key)

KeyError: 0

```

```

[50]: # =====
# Obtener la primera fila utilizando iloc
# =====

print(df_data.iloc[0])

```

```

ciudad          Valencia
temperatura      10
o2              1.0
humedad         NaN
co2             NaN
Name: a, dtype: object

```

```

[51]: # =====
# Acceder utilizando el índice semántico
# =====

display(df_data.loc["a"])

display(df_data.loc[:"b"])

```

```

ciudad      Valencia
temperatura      10
o2              1.0
humedad         NaN
co2             NaN
Name: a, dtype: object

```

```

      ciudad  temperatura  o2  humedad  co2
a  Valencia           10  1.0      NaN  NaN
b  Barcelona           8  NaN      NaN  NaN

```

```

[52]: # =====
# Seleccionar varias filas y columnas
# =====

display(
    df_data.loc[:"b"].loc[:, ["o2", "humedad"]]
)

```

```

      o2  humedad
a  1.0      NaN
b  NaN      NaN

```

```

[53]: # =====
# Diferencia entre referencia y copia
# =====

display(df_data)

# Obtener una referencia de una fila.
serie = df_data.loc["a"]

print(serie)

# Modificar la referencia también modifica el DataFrame.
serie.iloc[2] = 3000

display(df_data)

# Crear una copia independiente.
df_copia = df_data.loc["a"].copy()

df_copia.iloc[2] = 9999

display(df_copia)

display(df_data)

```

```

      ciudad  temperatura  o2  humedad  co2

```

a	Valencia	10	1.0	NaN	NaN
b	Barcelona	8	NaN	NaN	NaN
c	Valencia	9	NaN	NaN	NaN
d	Madrid	10	NaN	80.0	NaN
e	Sevilla	15	NaN	50.0	6.0
f	Valencia	10	NaN	90.0	10.0

```
ciudad      Valencia
temperatura      10
o2              1.0
humedad         NaN
co2            NaN
```

Name: a, dtype: object

	ciudad	temperatura	o2	humedad	co2
a	Valencia	10	1.0	NaN	NaN
b	Barcelona	8	NaN	NaN	NaN
c	Valencia	9	NaN	NaN	NaN
d	Madrid	10	NaN	80.0	NaN
e	Sevilla	15	NaN	50.0	6.0
f	Valencia	10	NaN	90.0	10.0

```
ciudad      Valencia
temperatura      10
o2           9999
humedad         NaN
co2            NaN
```

Name: a, dtype: object

	ciudad	temperatura	o2	humedad	co2
a	Valencia	10	1.0	NaN	NaN
b	Barcelona	8	NaN	NaN	NaN
c	Valencia	9	NaN	NaN	NaN
d	Madrid	10	NaN	80.0	NaN
e	Sevilla	15	NaN	50.0	6.0
f	Valencia	10	NaN	90.0	10.0

```
[54]: # =====
# Acceso utilizando axis
# =====

display(df_data.loc(axis=1)["ciudad"])
```

```
a      Valencia
b      Barcelona
c      Valencia
d      Madrid
e      Sevilla
f      Valencia
Name: ciudad, dtype: str
```

```
[55]: # =====
# Diferencia entre loc e iloc
# =====

frame = pd.DataFrame(
    {
        "Name": ["Carlos", "Pedro"],
        "Age": [34, 22]
    },
    index=[1, 0]
)

display(frame)

print("Primera fila\n")
print(frame.iloc[0])

print("\nElemento cuyo índice es 0\n")
print(frame.loc[0])
```

```
      Name  Age
1  Carlos   34
0  Pedro   22
```

Primera fila

```
Name    Carlos
Age       34
Name: 1, dtype: object
```

Elemento cuyo índice es 0

```
Name    Pedro
Age       22
Name: 0, dtype: object
```

10.0.1 Observaciones

- `iloc[]` trabaja con posiciones numéricas.
- `loc[]` trabaja con los nombres de los índices y columnas.
- Al seleccionar una parte del DataFrame normalmente se obtiene una **referencia**, por lo que los cambios pueden afectar al DataFrame original.
- Si se desea modificar los datos sin alterar el DataFrame original, es recomendable utilizar el método `.copy()`.
- Cuando existe ambigüedad entre índices numéricos y posiciones, siempre es preferible utilizar `loc` o `iloc` explícitamente.

10.1 4.4 Objeto Index de Pandas

Los objetos **Index** representan los índices utilizados por las Series y los DataFrames.

10.1.1 Características principales

- Permiten identificar de forma única filas o columnas.
- Son estructuras optimizadas para realizar búsquedas rápidas.
- Son **inmutables**, es decir, una vez creados no pueden modificarse directamente.
- Pueden utilizarse al crear un DataFrame personalizado.

```
[56]: # =====  
# Construcción de un objeto Index  
# =====  
  
import pandas as pd  
  
# Crear un objeto Index con varios valores enteros.  
ind = pd.Index([2, 3, 5, 23, 26])  
  
# Mostrar el cuarto elemento (posición 3).  
print(ind[3])  
  
# Mostrar elementos desde el inicio tomando uno cada dos posiciones.  
print(ind[::2])
```

23

Index([2, 5, 26], dtype='int64')

```
[57]: # =====  
# Utilizar un objeto Index al crear un DataFrame  
# =====  
  
# Crear un DataFrame utilizando el objeto Index como índice.  
frame = pd.DataFrame(  
    {  
        "Name": ["Carlos", "Pedro", "Manolo", "Luis", "Alberto"],  
        "Age": [34, 22, 15, 55, 23]  
    },  
    index=ind  
)  
  
display(frame)
```

	Name	Age
2	Carlos	34
3	Pedro	22
5	Manolo	15
23	Luis	55
26	Alberto	23

```
[58]: # =====
# Los objetos Index son inmutables
# =====

try:
    # Intentar modificar un elemento del índice.
    ind[3] = 8
except TypeError as e:
    print("Error:", e)
```

Error: Index does not support mutable operations

```
[59]: # =====
# Cambiar la columna utilizada como índice
# =====

# Crear nuevamente el DataFrame.
frame = pd.DataFrame(
    {
        "Name": ["Carlos", "Pedro", "Manolo", "Luis", "Alberto"],
        "Age": [34, 22, 15, 55, 23]
    },
    index=ind
)

print("DataFrame original:")
display(frame)

# Convertir la columna 'Age' en el nuevo índice.
frame.set_index("Age", inplace=True)

print("DataFrame con Age como índice:")
display(frame)
```

DataFrame original:

	Name	Age
2	Carlos	34
3	Pedro	22
5	Manolo	15
23	Luis	55
26	Alberto	23

DataFrame con Age como índice:

	Name
Age	
34	Carlos
22	Pedro
15	Manolo

55 Luis
23 Alberto

11 4.5 Slicing

El **slicing** permite seleccionar una parte del DataFrame.

Se puede realizar mediante:

- Filas.
- Columnas.
- Índices semánticos (`loc`).
- Índices posicionales (`iloc`).
- Condiciones lógicas.

```
[60]: # =====  
# Crear un DataFrame para los ejemplos  
# =====  
  
d_and_d_characters = {  
    "Name": ["bundenth", "theorin", "barlok"],  
    "Strength": [10, 12, 19],  
    "Wisdom": [20, 13, 6]  
}  
  
character_data = pd.DataFrame(  
    d_and_d_characters,  
    index=["a", "b", "c"]  
)  
  
display(character_data)  
  
# Mostrar todas las filas excepto la última.  
display(character_data[:-1])  
  
# Mostrar únicamente la segunda fila.  
display(character_data[1:2])
```

	Name	Strength	Wisdom
a	bundenth	10	20
b	theorin	12	13
c	barlok	19	6

	Name	Strength	Wisdom
a	bundenth	10	20
b	theorin	12	13

	Name	Strength	Wisdom
b	theorin	12	13

```
[61]: # =====
# Seleccionar columnas específicas
# =====

display(character_data[["Name", "Wisdom"]])
```

	Name	Wisdom
a	bundenth	20
b	theorin	13
c	barlok	6

```
[62]: # =====
# Slicing utilizando iloc y loc
# =====

# iloc trabaja con posiciones.
display(character_data.iloc[1:])

# loc trabaja con nombres de índices y columnas.
display(character_data.loc['b', 'Name':'Strength'])
```

	Name	Strength	Wisdom
b	theorin	12	13
c	barlok	19	6

	Name	Strength
a	bundenth	10
b	theorin	12

```
[63]: # =====
# Obtener únicamente las columnas Name y Strength
# =====

display(character_data.loc[:, "Name":"Strength"])
```

	Name	Strength
a	bundenth	10
b	theorin	12
c	barlok	19

```
[64]: # =====
# Seleccionar filas y columnas específicas con loc
# =====

display(
    character_data.loc[
        ["a", "c"],
        ["Name", "Wisdom"]
    ]
)
```

```
)
```

	Name	Wisdom
a	bundenth	20
c	barlok	6

```
[65]: # =====  
# Seleccionar filas y columnas específicas con iloc  
# =====  
  
# Filas 0 y 2, columnas 0 y 2.  
display(character_data.iloc[[0, 2], [0, 2]])  
  
# También es posible utilizar índices negativos.  
display(character_data.iloc[[0, -1], [0, -1]])
```

	Name	Wisdom
a	bundenth	20
c	barlok	6

	Name	Wisdom
a	bundenth	20
c	barlok	6

```
[66]: # =====  
# Filtrar personajes con Strength mayor que 11  
# =====  
  
display(  
    character_data.loc[  
        character_data["Strength"] > 11,  
        ["Name", "Strength"]  
    ]  
)
```

	Name	Strength
b	theorin	12
c	barlok	19

```
[67]: # =====  
# Filtrar personajes con Strength > 15 o Wisdom > 15  
# =====  
  
display(  
    character_data.loc[  
        (character_data["Strength"] > 15) |  
        (character_data["Wisdom"] > 15)  
    ]  
)
```

	Name	Strength	Wisdom
a	bundenth	10	20
c	barlok	19	6

12 5 Cargar y guardar datos en Pandas

Pandas permite guardar y cargar datos desde distintos formatos de archivo, siendo **CSV** uno de los más utilizados.

Entre las principales funciones se encuentran:

- `to_csv()`: guarda un DataFrame en un archivo CSV.
- `read_csv()`: carga un archivo CSV dentro de un DataFrame.

Además, Pandas puede trabajar con otros formatos como:

- JSON
- Excel
- HDF5
- Parquet

Esto facilita el intercambio y almacenamiento de información para su posterior análisis.

```
[68]: # =====
# Guardar un DataFrame en un archivo CSV
# =====

import os

# Crear la ruta donde se almacenará el archivo.
ruta = os.path.join("res", "o_d_d_characters.csv")

# Guardar el DataFrame en formato CSV.
# El separador puede modificarse mediante el parámetro sep.
# En este caso se utiliza ';' como separador.
character_data.to_csv(ruta, sep=';')

print("Archivo CSV guardado correctamente.")
```

Archivo CSV guardado correctamente.

```
[69]: # =====
# Leer un archivo CSV
# =====

# Leer el archivo utilizando el mismo separador con el que fue guardado.
loaded = pd.read_csv(ruta, sep=';')

# Mostrar el contenido del DataFrame cargado.
display(loaded)
```

Unnamed: 0	Name	Strength	Wisdom
0	a bundenth	10	20
1	b theorin	12	13
2	c barlok	19	6

```
[70]: # =====
# Definir la ruta del archivo Titanic
# =====

ruta = os.path.join("res", "titanic.csv")
```

```
[71]: # =====
# Leer el conjunto de datos Titanic
# =====

# Cargar el archivo CSV utilizando coma como separador.
titanic = pd.read_csv(ruta, sep=",")

# Mostrar el DataFrame.
display(titanic)
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	
..	...	...	...	
886	887	0	2	
887	888	1	1	
888	889	0	3	
889	890	1	1	
890	891	0	3	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	
..	...	...	...	...	
886	Montvila, Rev. Juozas	male	27.0	0	
887	Graham, Miss. Margaret Edith	female	19.0	0	
888	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	
889	Behr, Mr. Karl Howell	male	26.0	0	
890	Dooley, Mr. Patrick	male	32.0	0	

Parch	Ticket	Fare	Cabin	Embarked
-------	--------	------	-------	----------

0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S
..	...	...	...	...	...
886	0	211536	13.0000	NaN	S
887	0	112053	30.0000	B42	S
888	2	W./C. 6607	23.4500	NaN	S
889	0	111369	30.0000	C148	C
890	0	370376	7.7500	NaN	Q

[891 rows x 12 columns]

```
[72]: # =====
# Leer un CSV utilizando una columna como índice
# =====

# index_col indica qué columna será utilizada como índice.
loaded = pd.read_csv(
    ruta,
    sep=";",
    index_col=0
)

display(loaded)
```

	Survived	Pclass	\
PassengerId			
1	0	3	
2	1	1	
3	1	3	
4	1	1	
5	0	3	
...	...	...	
887	0	2	
888	1	1	
889	0	3	
890	1	1	
891	0	3	

	Name	Sex	Age	\
PassengerId				
1	Braund, Mr. Owen Harris	male	22.0	
2	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	
3	Heikkinen, Miss. Laina	female	26.0	
4	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	
5	Allen, Mr. William Henry	male	35.0	

```

...
887                                Montvila, Rev. Juozas      male  27.0
888                        Graham, Miss. Margaret Edith  female  19.0
889                Johnston, Miss. Catherine Helen "Carrie"  female   NaN
890                        Behr, Mr. Karl Howell      male  26.0
891                        Dooley, Mr. Patrick      male  32.0

```

```

                SibSp  Parch                Ticket    Fare Cabin Embarked
PassengerId
1                1      0          A/5 21171    7.2500   NaN        S
2                1      0            PC 17599   71.2833   C85        C
3                0      0  STON/O2. 3101282    7.9250   NaN        S
4                1      0            113803   53.1000  C123        S
5                0      0            373450    8.0500   NaN        S
...
887                0      0            211536   13.0000   NaN        S
888                0      0            112053   30.0000   B42        S
889                1      2      W./C. 6607   23.4500   NaN        S
890                0      0            111369   30.0000  C148        C
891                0      0            370376    7.7500   NaN        Q

```

[891 rows x 11 columns]

12.1 Argumentos útiles de `to_csv()`

Al guardar un `DataFrame`, algunos parámetros útiles son:

- `sep`: define el separador de columnas.
- `index`: indica si se guarda el índice (`True` por defecto).
- `na_rep`: reemplaza los valores `NaN` por un texto específico.
- `encoding`: define la codificación del archivo (por ejemplo, `"utf-8"`).

12.1.1 Ejemplo

```
df.to_csv("datos.csv", sep=";", na_rep="Sin dato")
```

12.2 Argumentos útiles de `read_csv()`

Al cargar un archivo CSV, algunos parámetros frecuentes son:

- `sep`: especifica el separador de columnas.
- `index_col`: selecciona la columna que actuará como índice.
- `na_values`: indica qué valores deben interpretarse como `NaN`.
- `encoding`: especifica la codificación del archivo.

12.2.1 Ejemplo

```
df = pd.read_csv(
    "datos.csv",
```

```
    sep=";",  
    na_values="Sin dato"  
)
```

Pandas también proporciona funciones para trabajar con otros formatos de almacenamiento como:

- JSON
- Excel
- HDF5
- Parquet
- SQL

Estos formatos permiten intercambiar información entre diferentes aplicaciones y proyectos de análisis de datos.

13 6 Ejemplo práctico en Pandas

En esta sección se trabaja con el conjunto de datos **MovieLens 1M**, una de las bases de datos más utilizadas para aprender análisis de datos y sistemas de recomendación.

El conjunto de datos contiene aproximadamente un millón de valoraciones realizadas por usuarios sobre diferentes películas.

Además de las valoraciones, incluye información como:

- Usuarios.
- Género.
- Edad.
- Ocupación.
- Código postal.

A lo largo de este ejemplo se aprenderá a:

- Descargar un conjunto de datos desde Internet.
- Descomprimir archivos ZIP.
- Cargar archivos utilizando Pandas.
- Analizar estadísticas descriptivas.
- Filtrar información.
- Corregir datos.
- Agrupar registros.
- Guardar nuevamente el conjunto de datos procesado.

```
[73]: # =====  
# Importar las librerías necesarias  
# =====  
  
import numpy as np  
import pandas as pd  
import zipfile  
import urllib.request
```

```

import os

# URL del conjunto de datos MovieLens.
url = "http://files.grouplens.org/datasets/movielens/ml-1m.zip"

# Ruta donde se descargará el archivo ZIP.
ruta = os.path.join("res", "ml-1m.zip")

# Descargar el archivo.
urllib.request.urlretrieve(url, ruta)

print("Dataset descargado correctamente.")

```

Dataset descargado correctamente.

```

[74]: # =====
# Descomprimir el archivo ZIP descargado
# =====

ruta_ext = os.path.join("res")

with zipfile.ZipFile(ruta, "r") as z:

    print("Extrayendo archivos...")

    z.extractall(ruta_ext)

    print("Proceso finalizado.")

```

Extrayendo archivos...

Proceso finalizado.

```

[75]: # =====
# Leer el archivo users.dat
# =====

ruta_users = os.path.join("res", "ml-1m", "users.dat")

users_dataset = pd.read_csv(
    ruta_users,
    sep=":",
    index_col=0,
    engine="python"
)

display(users_dataset)

```

F 1.1 10 48067

1

```

2      M   56  16  70072
3      M   25  15  55117
4      M   45   7  02460
5      M   25  20  55455
6      F   50   9  55117
... .. ... .. ...
6036   F   25  15  32603
6037   F   45   1  76006
6038   F   56   1  14706
6039   F   45   0  01060
6040   M   25   6  11106

```

[6039 rows x 4 columns]

```

[76]: # =====
# Leer el archivo users.dat
# =====

ruta_users = os.path.join("res", "ml-1m", "users.dat")

users_dataset = pd.read_csv(
    ruta_users,
    sep=":",
    index_col=0,
    engine="python"
)

display(users_dataset)

```

```

      F  1.1  10  48067
1
2      M   56  16  70072
3      M   25  15  55117
4      M   45   7  02460
5      M   25  20  55455
6      F   50   9  55117
... .. ... .. ...
6036   F   25  15  32603
6037   F   45   1  76006
6038   F   56   1  14706
6039   F   45   0  01060
6040   M   25   6  11106

```

[6039 rows x 4 columns]

13.0.1 Observación

Al cargar el archivo anterior se presentan varios problemas:

- El archivo no contiene encabezados.
- La primera fila se interpreta incorrectamente.
- Las columnas aparecen sin nombres.

Por ello es necesario indicar manualmente los nombres de las columnas.

```
[77]: # =====
# Leer correctamente el archivo indicando los nombres
# de las columnas
# =====

users_dataset = pd.read_csv(
    ruta_users,
    sep="::",
    index_col=0,
    header=None,
    names=[
        "UserID",
        "Gender",
        "Age",
        "Occupation",
        "Zip-code"
    ],
    engine="python"
)

display(users_dataset)
```

	Gender	Age	Occupation	Zip-code
UserID				
1	F	1	10	48067
2	M	56	16	70072
3	M	25	15	55117
4	M	45	7	02460
5	M	25	20	55455
...	...	...	...	...
6036	F	25	15	32603
6037	F	45	1	76006
6038	F	56	1	14706
6039	F	45	0	01060
6040	M	25	6	11106

[6040 rows x 4 columns]

```
[78]: # =====
# Mostrar una muestra aleatoria del DataFrame
# =====
```

```
display(users_dataset.sample(10))
```

	Gender	Age	Occupation	Zip-code
UserID				
785	M	18	19	29307
5424	F	35	2	80012
3762	M	50	6	11746
4828	F	18	20	60194
5006	M	35	3	48827
2779	M	35	7	10040
530	M	25	2	10019
1298	M	35	6	33615
4069	M	35	15	19406
1279	M	35	1	98502

```
[79]: # =====  
# Mostrar las primeras filas  
# =====  
  
display(users_dataset.head(4))
```

	Gender	Age	Occupation	Zip-code
UserID				
1	F	1	10	48067
2	M	56	16	70072
3	M	25	15	55117
4	M	45	7	02460

```
[80]: # =====  
# Mostrar las últimas filas  
# =====  
  
display(users_dataset.tail(10))
```

	Gender	Age	Occupation	Zip-code
UserID				
6031	F	18	0	45123
6032	M	45	7	55108
6033	M	50	13	78232
6034	M	25	14	94117
6035	F	25	1	78734
6036	F	25	15	32603
6037	F	45	1	76006
6038	F	56	1	14706
6039	F	45	0	01060
6040	M	25	6	11106

```
[81]: # =====
# Mostrar las últimas filas
# =====

display(users_dataset.tail(10))
```

	Gender	Age	Occupation	Zip-code
UserID				
6031	F	18	0	45123
6032	M	45	7	55108
6033	M	50	13	78232
6034	M	25	14	94117
6035	F	25	1	78734
6036	F	25	15	32603
6037	F	45	1	76006
6038	F	56	1	14706
6039	F	45	0	01060
6040	M	25	6	11106

```
[82]: # =====
# Buscar códigos postales con más de cinco caracteres
# =====

display(
    users_dataset[
        users_dataset["Zip-code"].str.len() > 5
    ]
)
```

	Gender	Age	Occupation	Zip-code
UserID				
161	M	45	16	98107-2117
233	F	45	20	37919-4204
293	M	56	1	55337-4056
458	M	50	16	55405-2546
506	M	25	16	55103-1006
...	...	...	...	...
5682	M	18	0	23455-4959
5904	F	45	12	954025
5925	F	25	0	90035-4444
5967	M	50	16	73069-5429
5985	F	18	4	78705-5221

[81 rows x 4 columns]

```
[83]: # =====
# Obtener estadísticas descriptivas
# =====
```

```
display(users_dataset.describe())
```

	Age	Occupation
count	6040.000000	6040.000000
mean	30.639238	8.146854
std	12.895962	6.329511
min	1.000000	0.000000
25%	25.000000	3.000000
50%	25.000000	7.000000
75%	35.000000	14.000000
max	56.000000	20.000000

```
[84]: # =====
# Mostrar información general del DataFrame
# =====
```

```
users_dataset.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6040 entries, 1 to 6040
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Gender      6040 non-null   str
1   Age         6040 non-null   int64
2   Occupation  6040 non-null   int64
3   Zip-code    6040 non-null   str
dtypes: int64(2), str(2)
memory usage: 188.9 KB
```

```
[86]: # =====
# Mostrar estadísticas incluyendo columnas no numéricas
# =====
```

```
display(
    users_dataset.describe(include="all")
)
```

	Gender	Age	Occupation	Zip-code
count	6040	6040.000000	6040.000000	6040
unique	2	NaN	NaN	3439
top	M	NaN	NaN	48104
freq	4331	NaN	NaN	19
mean	NaN	30.639238	8.146854	NaN
std	NaN	12.895962	6.329511	NaN
min	NaN	1.000000	0.000000	NaN
25%	NaN	25.000000	3.000000	NaN
50%	NaN	25.000000	7.000000	NaN

75%	NaN	35.000000	14.000000	NaN
max	NaN	56.000000	20.000000	NaN

```
[87]: # =====
# Contar cuántas usuarias son mujeres
# =====

cantidad_mujeres = len(
    users_dataset[
        users_dataset["Gender"] == "F"
    ]
)

print("Cantidad de mujeres:", cantidad_mujeres)
```

Cantidad de mujeres: 1709

```
[88]: # =====
# Mostrar únicamente los usuarios menores de edad
# =====

under_age = users_dataset[
    users_dataset["Age"] == 1
]

print(len(under_age))

display(under_age.sample(10))
```

222

	Gender	Age	Occupation	Zip-code
UserID				
3382	F	1	10	55305
884	M	1	10	49454
2133	F	1	10	01607
2176	M	1	19	73505
875	M	1	10	94707
1878	M	1	10	92846
4807	M	1	2	04101
1568	F	1	10	94086
1478	M	1	10	92620
5390	M	1	0	78130

13.1 Modificación incorrecta de un DataFrame

El siguiente ejemplo produce el conocido aviso **SettingWithCopyWarning**.

Se incluye únicamente con fines didácticos para mostrar una práctica que debe evitarse.

```
[90]: # =====
# Ejemplo que genera SettingWithCopyWarning
# =====

users_dataset = pd.read_csv(
    ruta_users,
    sep="::",
    index_col=0,
    header=None,
    names=["UserID", "Gender", "Age", "Occupation", "Zip-code"],
    engine="python"
)

# Convertir la columna Age para permitir valores NaN.
users_dataset["Age"] = users_dataset["Age"].astype(float)

under_age = users_dataset[
    users_dataset["Age"] == 1
]

# Esta operación genera el SettingWithCopyWarning.
under_age.loc[:, "Age"] = np.nan

display(under_age.head())
```

	Gender	Age	Occupation	Zip-code
UserID				
1	F	NaN	10	48067
19	M	NaN	10	48073
51	F	NaN	10	10562
75	F	NaN	10	01748
86	F	NaN	10	54467

```
[91]: # =====
# Solución utilizando copy()
# =====

users_dataset = pd.read_csv(
    ruta_users,
    sep="::",
    index_col=0,
    header=None,
    names=["UserID", "Gender", "Age", "Occupation", "Zip-code"],
    engine="python"
)

under_age = users_dataset[
    users_dataset["Age"] == 1
]
```

```

]

under_age_copy = under_age.copy()

under_age_copy["Age"] = np.nan

display(under_age_copy.head())

users_dataset.loc[
    users_dataset["Age"] == 1
] = under_age_copy

display(users_dataset.head())

```

	Gender	Age	Occupation	Zip-code
UserID				
1	F	NaN	10	48067
19	M	NaN	10	48073
51	F	NaN	10	10562
75	F	NaN	10	01748
86	F	NaN	10	54467

	Gender	Age	Occupation	Zip-code
UserID				
1	F	NaN	10	48067
2	M	56.0	16	70072
3	M	25.0	15	55117
4	M	45.0	7	02460
5	M	25.0	20	55455

```

[92]: # =====
# Eliminar definitivamente los registros con edades inválidas
# =====

users_dataset = pd.read_csv(
    ruta_users,
    sep="::",
    index_col=0,
    header=None,
    names=["UserID", "Gender", "Age", "Occupation", "Zip-code"],
    engine="python"
)

# Reemplazar las edades incorrectas por NaN.
users_dataset.loc[
    users_dataset["Age"] == 1,
    "Age"
] = np.nan

```

```

display(
    users_dataset.loc[
        pd.isnull(users_dataset["Age"])
    ].head(4)
)

# Eliminar los registros con NaN.
users_dataset.drop(
    users_dataset[
        pd.isnull(users_dataset["Age"])
    ].index,
    inplace=True
)

display(users_dataset.head())

```

	Gender	Age	Occupation	Zip-code
UserID				
1	F	NaN	10	48067
19	M	NaN	10	48073
51	F	NaN	10	10562
75	F	NaN	10	01748

	Gender	Age	Occupation	Zip-code
UserID				
2	M	56.0	16	70072
3	M	25.0	15	55117
4	M	45.0	7	02460
5	M	25.0	20	55455
6	F	50.0	9	55117

```

[93]: # =====
# Agrupar usuarios por género y obtener estadísticas
# =====

display(
    users_dataset.groupby(
        by="Gender"
    ).describe()
)

```

	Age								Occupation	\
	count	mean	std	min	25%	50%	75%	max	count	
Gender										
F	1631.0	32.287554	11.792015	18.0	25.0	25.0	45.0	56.0	1631.0	
M	4187.0	31.568665	11.716053	18.0	25.0	25.0	35.0	56.0	4187.0	

	mean	std	min	25%	50%	75%	max
Gender							
F	6.498467	5.960285	0.0	1.0	4.0	11.0	20.0
M	8.743253	6.441753	0.0	4.0	7.0	15.0	20.0

```
[94]: # =====
# Guardar el DataFrame procesado
# =====

ruta_output = os.path.join(
    "res",
    "ml-1m",
    "o_users_processed.csv"
)

users_dataset.to_csv(
    ruta_output,
    sep=",",
    na_rep="null"
)

print("Archivo guardado correctamente.")
```

Archivo guardado correctamente.

14 7 Ejercicios

En esta sección se realiza un pequeño análisis sobre el conjunto de datos **MovieLens 1M**, utilizando el archivo **ratings.dat**.

Los objetivos son:

- Cargar el archivo de valoraciones.
- Comprobar si existen usuarios que no hayan realizado ninguna valoración.
- Identificar cuántos usuarios tienen menos de 30 reseñas registradas.

```
[95]: # =====
# Cargar el archivo ratings.dat
# =====

import os
import pandas as pd

# Construir la ruta del archivo.
ruta_ratings = os.path.join("res", "ml-1m", "ratings.dat")

# Leer el archivo indicando que utiliza "::" como separador
# y asignando nombres a las columnas.
ratings_dataset = pd.read_csv(
```

```

ruta_ratings,
sep="::",
header=None,
names=[
    "UserID",
    "MovieID",
    "Rating",
    "Timestamp"
],
engine="python"
)

# Mostrar el contenido del DataFrame.
display(ratings_dataset)

```

	UserID	MovieID	Rating	Timestamp
0	1	1193	5	978300760
1	1	661	3	978302109
2	1	914	3	978301968
3	1	3408	4	978300275
4	1	2355	5	978824291
...	...	...	...	...
1000204	6040	1091	1	956716541
1000205	6040	1094	5	956704887
1000206	6040	562	5	956704746
1000207	6040	1096	4	956715648
1000208	6040	1097	4	956715569

[1000209 rows x 4 columns]

```

[96]: # =====
# Verificar si existen usuarios sin ninguna valoración
# =====

# Contar cuántos usuarios aparecen en el archivo de ratings.
n_users_with_reviews = len(
    ratings_dataset.groupby(by="UserID")
)

print("Usuarios con valoraciones:", n_users_with_reviews)

# Número total de usuarios del dataset users.dat.
n_users = len(users_dataset)

print("Total de usuarios:", n_users)

# Calcular cuántos usuarios no realizaron ninguna valoración.

```

```
print(
    "Usuarios sin valoraciones:",
    n_users - n_users_with_reviews
)
```

Usuarios con valoraciones: 6040

Total de usuarios: 5818

Usuarios sin valoraciones: -222

```
[97]: # =====
# Usuarios con menos de 30 valoraciones
# =====

# Agrupar las valoraciones por usuario.
# Posteriormente se filtran únicamente aquellos usuarios
# que tienen menos de 30 reseñas registradas.
ratings_under_30 = (
    ratings_dataset
    .groupby("UserID")["UserID"]
    .filter(lambda x: len(x) < 30)
    .value_counts()
)

# Mostrar el resultado.
display(ratings_under_30)

# Mostrar cuántos usuarios cumplen esta condición.
print(
    "Usuarios con menos de 30 reviews:",
    len(ratings_under_30)
)
```

```
UserID
71      29
221     29
234     29
246     29
282     29
..
5533    20
5725    20
5828    20
5904    20
6038    20
Name: count, Length: 751, dtype: int64

Usuarios con menos de 30 reviews: 751
```

14.1 Conclusión

Durante esta práctica se utilizaron las principales funcionalidades de la librería **Pandas** para el análisis de datos.

Se aprendió a:

- Crear y manipular Series y DataFrames.
- Filtrar, indexar y modificar información.
- Leer y guardar archivos CSV.
- Analizar estadísticas descriptivas.
- Agrupar datos por categorías.
- Trabajar con un conjunto de datos real (MovieLens 1M).

Estas herramientas constituyen la base del análisis de datos y del preprocesamiento utilizado posteriormente en proyectos de Ciencia de Datos y Machine Learning.

[]: